

Universidad Europea de Madrid

Laboratorio de Hacking

Practica 3 - Mitm y suplantacion

Autor/a: Adrian Garcia Mejias

Titulacion: Grado en Ingenieria Informatica

Asignatura: Laboratorio de Hacking

Fecha de entrega: 26 de abril

Resumen

Este trabajo estudia, en un pequeño laboratorio propio y aislado (nunca contra sistemas reales de terceros), dos formas en las que alguien podría «colarse» en medio de una conversación entre dos ordenadores sin que estos se den cuenta, y como detectar cuando esto está ocurriendo.

La primera parte trata sobre como los dispositivos de una misma red local se encuentran entre sí. Cuando un ordenador quiere hablar con otro de su misma red, pregunta en voz alta «quien tiene esta dirección» y confía ciegamente en la primera respuesta que recibe. Esto permite que alguien malintencionado responda antes que el dispositivo legítimo, haciéndose pasar por él, y consiguiendo que todo el tráfico pase primero por su ordenador antes de llegar a su destino real – como si alguien suplantara la identidad del cartero para leer el correo de otra persona antes de entregarlo. Se ha construido un pequeño programa capaz de hacer exactamente esto de forma controlada, y otro programa distinto capaz de detectarlo: si una misma dirección «responde» desde dos identidades distintas en poco tiempo, es una señal clara de que algo sospechoso está pasando.

La segunda parte trata sobre el sistema que traduce nombres de páginas web (como «www.ejemplo.com») en las direcciones numéricas que realmente usan los ordenadores para comunicarse. Un atacante puede aprovechar este sistema de dos formas: intentando «adivinar» respuestas falsas antes de que llegue la respuesta real (para engañar al sistema y redirigir a los usuarios a páginas falsas), o preguntando muy rápido por muchos nombres inventados para averiguar que sistemas existen dentro de una empresa sin que nadie se lo diga directamente. Se ha construido un sistema que detecta este patrón: si una misma dirección recibe demasiadas respuestas de «ese nombre no existe» en muy poco tiempo, salta una alerta.

Los resultados muestran que ambas técnicas de ataque, aunque técnicamente sencillas de llevar a cabo, dejan un rastro claro y detectable si se vigila el tráfico de red de la forma adecuada – la clave no está en impedir que estos ataques existan (los protocolos afectados llevan décadas en uso y son difíciles de cambiar), sino en saber reconocer sus síntomas a tiempo.

Indice

Resumen	2
1 Introduccion	4
2 Desarrollo	5
2.1 Deteccion de envenenamiento ARP	5
2.1.1 Topologia del laboratorio	5
2.1.2 Intento de uso de bettercap	5
2.1.3 Envenenamiento ARP manual con scapy	5
2.1.4 Deteccion con <code>alert_arpspoof</code>	6
2.2 Suplantacion y anomalias DNS	7
2.2.1 Contexto: el ataque de Kaminsky y el DNS snooping	7
2.2.2 Escenario de red	8
2.2.3 Deteccion con <code>alert_dnssnooping</code>	8
2.2.4 Validacion con generador de trafico en scapy	8
3 Resultados	9
3.1 ARP Spoofing	10
3.2 DNS Snooping / Kaminsky	10
4 Conclusiones	10
5 Bibliografia	11

1 Introduccion

Tras el reconocimiento pasivo (Practica 1) y el reconocimiento activo (Practica 2), esta tercera practica aborda un tipo de tecnica distinto: los ataques de **Man-In-The-Middle** (MITM) y suplantacion, centrados no en descubrir informacion sino en **interceptar o manipular** la comunicacion entre dos partes que confian mutuamente en la integridad de los protocolos de red que utilizan.

Se abordan dos vectores concretos, ambos explotando debilidades estructurales de protocolos sin autenticacion fuerte por diseno:

1. **ARP Spoofing:** el protocolo ARP (Address Resolution Protocol) no autentica sus respuestas, permitiendo a un atacante en la misma red local falsificar la asociacion IP-MAC de otros hosts y posicionarse como intermediario de su trafico.
2. **DNS Snooping y ataque de Kaminsky:** el protocolo DNS, en su forma clasica sin DNSSEC, tampoco autentica fuertemente sus respuestas mas alla de un identificador de transaccion facilmente adivinable en rafaga, permitiendo tanto el envenenamiento de cache (Kaminsky) como el mapeo de infraestructura interna mediante fuerza bruta de subdominios (DNS snooping).

El objetivo de esta practica no es unicamente ejecutar estos ataques, sino sobre todo **implementar sistemas de deteccion basados en firmas** capaces de identificarlos en tiempo real analizando el trafico de red, un enfoque tipico de un IDS (Intrusion Detection System) de red.

Tal como exige expresamente el enunciado de la asignatura, toda actividad de suplantacion y envenenamiento se ha limitado estrictamente a redes virtuales propias, desplegadas mediante **docker-compose** en contenedores aislados. En ningun momento se ha dirigido trafico de ataque hacia resolutores DNS publicos, ni se ha interceptado trafico fuera del entorno Docker configurado especificamente para esta practica.

Los objetivos concretos de esta practica son:

1. Definir un escenario de red en contenedores con una victima, un router y un servidor web en una red externa, y verificar su conectividad.
2. Investigar y utilizar **bettercap** para el envenenamiento de tablas ARP, documentando cualquier limitacion encontrada.
3. Implementar una funcion de deteccion (**alert_arpspoof**) que identifique anomalias en las respuestas ARP indicativas de envenenamiento.
4. Definir un segundo escenario de red con un resolver DNS y un servidor DNS autoritativo.
5. Implementar una funcion de deteccion (**alert_dnssnooping**) basada en umbral de volumen de respuestas NXDOMAIN.
6. Validar ambos sistemas de deteccion mediante scripts en scapy que generen el trafico de ataque correspondiente, evidenciando las alertas resultantes.

En ningun momento de esta practica se ha realizado ninguna actividad de suplantacion o envenenamiento contra sistemas ajenos al laboratorio Docker desplegado y controlado en su totalidad por el autor de este informe.

2 Desarrollo

2.1 Deteccion de envenenamiento ARP

2.1.1 Topologia del laboratorio

Se ha definido, mediante `docker-compose` [1], un escenario de red con tres contenedores en dos subredes distintas, simulando una topologia realista donde el trafico entre una victima y un servidor externo debe pasar obligatoriamente por un router intermedio:

- **Victima** (192.168.10.20): contenedor Alpine en la red interna `lan_interna` (192.168.10.0/24).
- **Router** (192.168.10.2 / 192.168.20.2): contenedor Alpine con reenvio IP activo (`sysctl: net.ipv4.ip_forward=1`), presente en ambas redes, actuando de puente entre la victima y el servidor.
- **Servidor web** (192.168.20.10): contenedor nginx en la red externa `lan_externa` (192.168.20.0/24).

Cada contenedor incorpora en su arranque la configuracion de rutas necesaria (`ip route add`) para que el trafico victima servidor fluya correctamente a traves del router, verificado mediante una prueba de conectividad extremo a extremo (`ping` con perdida de paquetes 0%) antes de proceder con el ataque. Esta topologia situa al atacante (la maquina Kali, conectada a la interfaz bridge de `lan_interna`) en la misma capa de enlace que la victima y el router, condicion necesaria para que un ataque de ARP spoofing sea posible, ya que el protocolo ARP no se enruta mas alla de la red local.

2.1.2 Intento de uso de bettercap

Tal como exige el enunciado, se investigo y se intento utilizar **bettercap** v2.41.5 [2] para iniciar el envenenamiento de las tablas ARP de la victima y el router. Tras multiples intentos — incluyendo la generacion de trafico ICMP continuo para mantener los hosts activos, la limpieza de la cache ARP de ambos extremos, y el ajuste del periodo de refresco de descubrimiento de host (`net.recon.period`) — la herramienta presento de forma reproducible el error `could not find spoof targets`, incluso confirmando mediante `net.probe` que si detectaba correctamente ambos objetivos (eventos `endpoint.new`), para perderlos segundos despues (`endpoint.lost`) en un ciclo repetido que impedia al modulo `arp.spoof` mantenerlos como objetivos validos.

Esta limitacion coincide con problemas reportados publicamente por otros usuarios de bettercap en entornos de red virtualizados o en contenedores (issues 1047, 979 y 942 del repositorio oficial del proyecto en GitHub), sin una solucion definitiva documentada a la fecha de esta practica. El registro completo de la investigacion y los comandos probados se adjunta en `bettercap_troubleshooting.txt`, en esta misma carpeta del repositorio.

2.1.3 Envenenamiento ARP manual con scapy

Dado que la limitacion identificada es de la propia herramienta y no del diseno del laboratorio (la conectividad ya se habia verificado como correcta), se opto por implementar el envenenamiento directamente con **scapy** [3], mediante el script `arp_spoof.py`. El script:

1. Resuelve las MAC reales de la victima y el router mediante una peticion ARP legitima (`srp` con `broadcast`).

2. Construye, para cada extremo, una trama Ethernet + ARP falsa (op=2, «is-at») que asocia la IP del otro extremo con la MAC del atacante, y la reenvia periodicamente (cada 2 segundos) para mantener el envenenamiento activo frente a la caducidad natural de las entradas ARP.
3. Al recibir una interrupcion (Ctrl+C), restaura las entradas ARP reales en ambos extremos, evitando dejar la red del laboratorio en un estado inconsistente.

Tras ejecutar el ataque, la inspeccion directa de las tablas ARP de ambos contenedores confirma el envenenamiento: tanto la victima como el router asocian la IP del otro extremo con la MAC del atacante (02:42:86:4c:b7:0b) en lugar de su MAC real (ver Figura 1).

```
(adr3d0@adr3d0)-[~]
$ sudo docker exec p3-router ip neigh
192.168.10.20 dev eth1 lladdr 02:42:86:4c:b7:0b ref 1 used 0/0/0 probes 1 REACHABLE
192.168.10.1 dev eth1 lladdr 02:42:86:4c:b7:0b used 0/0/0 probes 4 STALE
192.168.20.10 dev eth0 lladdr 02:42:c0:a8:14:0a used 0/0/0 probes 1 STALE

(adr3d0@adr3d0)-[~]
$ echo "=== Tabla ARP de la victima (192.168.10.20) ===" && sudo docker exec p3-victima ip neigh && echo "" && echo "=== Tabla ARP del router (192.168.10.2) ===" && sudo docker exec p3-router ip neigh
=== Tabla ARP de la victima (192.168.10.20) ===
192.168.10.1 dev eth0 lladdr 02:42:86:4c:b7:0b used 0/0/0 probes 4 STALE
192.168.10.2 dev eth0 lladdr 02:42:86:4c:b7:0b ref 1 used 0/0/0 probes 1 REACHABLE
=== Tabla ARP del router (192.168.10.2) ===
192.168.10.20 dev eth1 lladdr 02:42:86:4c:b7:0b ref 1 used 0/0/0 probes 1 REACHABLE
192.168.10.1 dev eth1 lladdr 02:42:86:4c:b7:0b used 0/0/0 probes 4 STALE
192.168.20.10 dev eth0 lladdr 02:42:c0:a8:14:0a used 0/0/0 probes 1 STALE

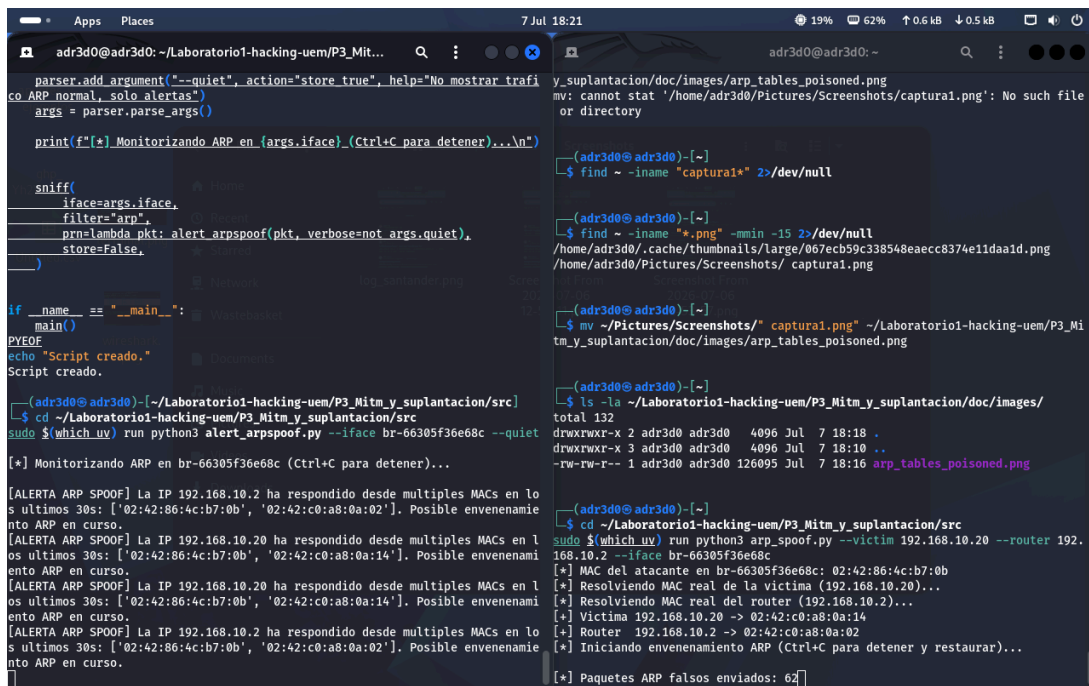
(adr3d0@adr3d0)-[~]
$
```

Figura 1: Tablas ARP de la victima y el router tras el ataque, mostrando la MAC del atacante suplantando a ambos extremos.

2.1.4 Deteccion con alert_arp spoof

Para detectar este tipo de ataque se ha implementado la funcion `alert_arp spoof` (archivo `alert_arp spoof.py`), que monitoriza pasivamente el trafico ARP de la interfaz del laboratorio mediante `sniff()` de `scapy`. La firma de deteccion se basa en el indicador mas fiable de ARP spoofing: **una misma direccion IP respondiendo (is-at, op=2) desde mas de una direccion MAC dentro de una ventana temporal reciente** (30 segundos en esta implementacion). En una red sana, cada IP mantiene una unica MAC estable; la aparicion de una segunda MAC para la misma IP es, con muy alta probabilidad, indicio de una respuesta ARP falsificada.

Al ejecutar `alert_arp spoof.py` en paralelo al ataque manual descrito en la seccion anterior, el detector genero alertas de forma inmediata y consistente para ambas IPs afectadas (ver Figura 2), confirmando tanto la eficacia del ataque como la correccion de la firma de deteccion implementada. Antes de iniciar el ataque, el detector no genero ninguna alerta, evidenciando ausencia de falsos positivos en condiciones de trafico ARP normal.



```
parser.add_argument("--quiet", action="store_true", help="No mostrar trafico ARP normal, solo alertas")
args = parser.parse_args()

print(f"[*] Monitorizando ARP en {args.iface} (Ctrl+C para detener)...")

sniff(
    iface=args.iface,
    filter="arp",
    prn=lambda pkt: alert_arpspoof(pkt, verbose=not args.quiet),
    store=False,
)

if __name__ == "__main__":
    main()
PYEOF
echo "Script creado."
Script creado.

[adr3d0@adr3d0:~/Laboratorio1-hacking-uem/P3_Mitm_y_suptancion/src]
$ cd ~/Laboratorio1-hacking-uem/P3_Mitm_y_suptancion/src
$ sudo $(which uv) run python3 alert_arpspoof.py --iface br-66305f36e68c --quiet

[*] Monitorizando ARP en br-66305f36e68c (Ctrl+C para detener)...

[ALERTA ARP SPOOF] La IP 192.168.10.2 ha respondido desde multiples MACs en los ultimos 30s: ['02:42:86:4c:b7:0b', '02:42:c0:a8:0a:02']. Posible envenenamiento ARP en curso.
[ALERTA ARP SPOOF] La IP 192.168.10.2 ha respondido desde multiples MACs en los ultimos 30s: ['02:42:86:4c:b7:0b', '02:42:c0:a8:0a:14']. Posible envenenamiento ARP en curso.
[ALERTA ARP SPOOF] La IP 192.168.10.2 ha respondido desde multiples MACs en los ultimos 30s: ['02:42:86:4c:b7:0b', '02:42:c0:a8:0a:14']. Posible envenenamiento ARP en curso.
[ALERTA ARP SPOOF] La IP 192.168.10.2 ha respondido desde multiples MACs en los ultimos 30s: ['02:42:86:4c:b7:0b', '02:42:c0:a8:0a:02']. Posible envenenamiento ARP en curso.

[adr3d0@adr3d0:~/Laboratorio1-hacking-uem/P3_Mitm_y_suptancion/src]
$ cd ~/Laboratorio1-hacking-uem/P3_Mitm_y_suptancion/src
$ sudo $(which uv) run python3 arp_spoof.py --victim 192.168.10.2 --router 192.168.10.2 --iface br-66305f36e68c
[*] MAC del atacante en br-66305f36e68c: 02:42:86:4c:b7:0b
[*] Resolviendo MAC real de la victima (192.168.10.2)...
[*] Resolviendo MAC real del router (192.168.10.2)...
[*] Victima 192.168.10.2 -> 02:42:c0:a8:0a:14
[*] Router 192.168.10.2 -> 02:42:c0:a8:0a:02
[*] Iniciando envenenamiento ARP (Ctrl+C para detener y restaurar)...

[*] Paquetes ARP falsos enviados: 62

[adr3d0@adr3d0:~/Laboratorio1-hacking-uem/P3_Mitm_y_suptancion/doc/images/arp_tables_poisoned.png]
mv: cannot stat '/home/adr3d0/Pictures/Screenshots/captura1.png': No such file or directory

[adr3d0@adr3d0:~/Laboratorio1-hacking-uem/P3_Mitm_y_suptancion/doc/images/arp_tables_poisoned.png]
$ find ~ -iname "captura1*" 2>/dev/null

[adr3d0@adr3d0:~/Laboratorio1-hacking-uem/P3_Mitm_y_suptancion/doc/images/arp_tables_poisoned.png]
$ find ~ -iname "*.png" -mmin -15 2>/dev/null
/home/adr3d0/.cache/thumbnails/large/067ecb59c338548eaec8374e11daa1d.png
/home/adr3d0/Pictures/Screenshots/captura1.png

[adr3d0@adr3d0:~/Laboratorio1-hacking-uem/P3_Mitm_y_suptancion/doc/images/arp_tables_poisoned.png]
$ mv ~/Pictures/Screenshots/captura1.png ~/Laboratorio1-hacking-uem/P3_Mitm_y_suptancion/doc/images/arp_tables_poisoned.png

[adr3d0@adr3d0:~/Laboratorio1-hacking-uem/P3_Mitm_y_suptancion/doc/images/arp_tables_poisoned.png]
$ ls -la ~/Laboratorio1-hacking-uem/P3_Mitm_y_suptancion/doc/images/
total 132
drwxrwxr-x 2 adr3d0 adr3d0 4096 Jul 7 18:18 .
drwxrwxr-x 3 adr3d0 adr3d0 4096 Jul 7 18:10 ..
-rw-rw-r-- 1 adr3d0 adr3d0 126095 Jul 7 18:16 arp_tables_poisoned.png
```

Figura 2: Alertas generadas por alert_arpspoof.py durante la ejecucion del ataque manual con scapy.

2.2 Suplantacion y anomalias DNS

2.2.1 Contexto: el ataque de Kaminsky y el DNS snooping

El **ataque de Kaminsky** (Dan Kaminsky, 2008) explota una debilidad estructural del protocolo DNS clasico: al no autenticar sus respuestas mas alla de un identificador de transaccion de 16 bits y un puerto de origen, un atacante puede enviar una rafaga masiva de consultas hacia un resolver por subdominios inexistentes de un dominio legitimo (por ejemplo, aleatorio1.banco.com, aleatorio2.banco.com...), y simultaneamente inundar al resolver con respuestas falsificadas que intentan adivinar el identificador de transaccion correcto antes de que llegue la respuesta real del servidor autoritativo. Si una respuesta falsificada acierta, el resolver la almacena en cache como legitima, envenenando su cache DNS para todos los clientes que lo consulten posteriormente – pudiendo redirigir trafico legitimo hacia infraestructura del atacante.

El **DNS snooping** (o mapeo de subdominios) comparte el mismo patron de trafico superficial – rafagas de consultas a subdominios inexistentes o poco comunes – pero con un objetivo distinto: no busca envenenar la cache, sino inferir la topologia interna de una organizacion observando que subdominios existen (respuesta positiva) frente a los que no (NXDOMAIN), o abusando de la cache compartida de un resolver para determinar si un dominio ha sido consultado recientemente por otros usuarios de la misma red.

Ambas tecnicas comparten una firma observable identica desde la perspectiva de deteccion: **un volumen anormalmente alto de consultas a subdominios inexistentes de un mismo dominio, en un periodo de tiempo corto**, lo que justifica implementar una deteccion basada en umbral sobre el volumen de respuestas NXDOMAIN, tal como exige el enunciado de esta practica.

2.2.2 Escenario de red

Se ha definido, mediante `docker-compose` [1], un segundo escenario con dos contenedores en la red `lan_dns` (192.168.30.0/24): un **servidor DNS** (192.168.30.10) con una zona propia (`lab.local`) y un **resolver DNS** (192.168.30.20) configurado para reenviar consultas al servidor autoritativo. Ambos se han desplegado con la imagen `cytopia/bind`, unica encontrada con soporte multi-arquitectura (incluyendo ARM64) entre las probadas.

Durante la configuracion del reenvio explicito (`forward only`) del resolver hacia el servidor autoritativo se encontro una limitacion del entypoint de la imagen, que regenera la configuracion desde variables de entorno en cada arranque, dificultando persistir una opcion no soportada nativamente por dichas variables. Se resolvio montando un `named.conf` propio y autosuficiente, arrancando `named` directamente sin pasar por el script de entrada de la imagen. Independientemente del mecanismo interno de resolucion, el resolver cumple la funcion necesaria para esta practica: responder de forma consistente con `NXDOMAIN` ante consultas a subdominios inexistentes, que es precisamente el comportamiento que la tecnica de Kaminsky/DNS snooping explota y que el sistema de deteccion implementado debe identificar.

2.2.3 Deteccion con `alert_dnssnooping`

Se ha implementado la funcion `alert_dnssnooping` (fichero `alert_dnssnooping.py`), que monitoriza pasivamente el trafico DNS de la interfaz del laboratorio mediante `sniff()` de scapy [3], filtrando por puerto UDP 53. La firma de deteccion implementada es la exigida por el enunciado: **deteccion por volumen (threshold) de respuestas NXDOMAIN recibidas por una misma IP dentro de una ventana temporal**. Concretamente, la funcion:

1. Analiza cada paquete DNS de respuesta (`qr=1`) con codigo de respuesta `rcode=3` (`NXDOMAIN`).
2. Mantiene, por IP destino de la respuesta, una lista de marcas de tiempo de respuestas `NXDOMAIN` recientes, descartando las que quedan fuera de la ventana configurada (por defecto, 10 segundos).
3. Si el numero de respuestas `NXDOMAIN` dentro de la ventana alcanza el umbral configurado (por defecto, 8), dispara una alerta y reinicia el contador de esa IP, evitando alertas repetidas por cada paquete adicional de la misma rafaga (gestion de falsos positivos por duplicacion de alertas).

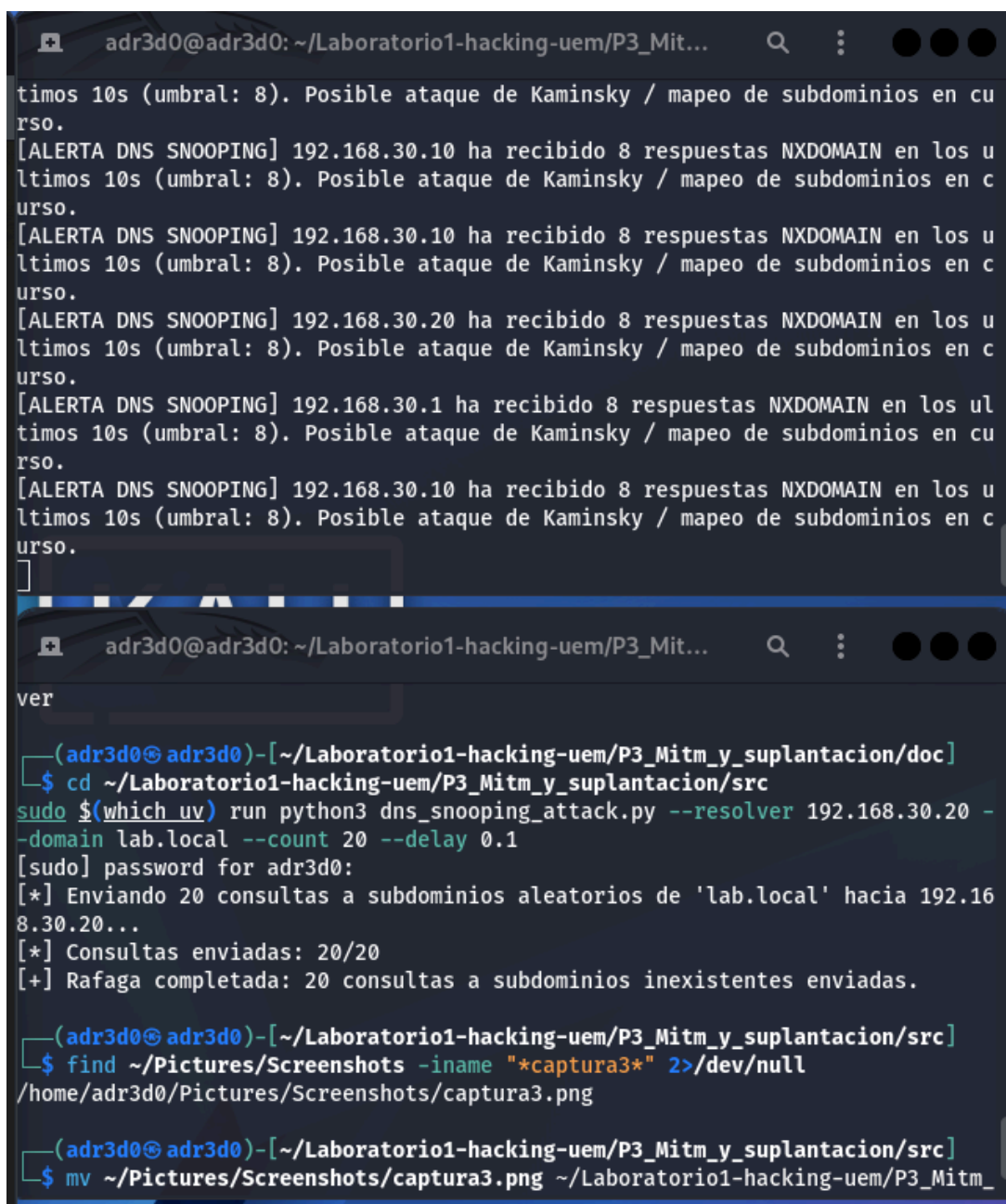
Los umbrales (`--threshold` y `--window`) son configurables por linea de comandos, permitiendo ajustar la sensibilidad del sistema segun el volumen de trafico DNS legitimo esperado en la red monitorizada – un umbral demasiado bajo generaria falsos positivos con trafico normal (errores tipograficos de usuarios, por ejemplo), mientras que uno demasiado alto podria no detectar rafagas de ataque mas lentas y sigilosas.

2.2.4 Validacion con generador de trafico en scapy

Para validar el sistema se ha implementado `dns_snooping_attack.py`, un script en scapy que genera una rafaga configurable de consultas DNS hacia subdominios aleatorios (8 caracteres alfanumericos) de un dominio base, simulando el patron de trafico caracteristico de Kaminsky/DNS snooping. Al ejecutar una rafaga de 20 consultas contra el resolver del laboratorio con `alert_dnssnooping.py` escuchando en paralelo (umbral: 8 `NXDOMAIN` / 10s), el sistema disparo multiples alertas de forma consistente durante la ejecucion del ataque (ver Figura 3).

Cabe destacar que, al capturar en modo promiscuo sobre la interfaz bridge compartida de Docker, el detector observa el trafico `NXDOMAIN` asociado a varias IPs de la topologia (el resolver, el servidor, y la propia maquina atacante), reflejo de como el trafico DNS de la rafaga

atraviesa multiples perspectivas de la red virtual del laboratorio. Este comportamiento no afecta a la validez de la deteccion: en cualquiera de las IPs monitorizadas, el volumen de NXDOMAIN generado por el ataque supera claramente el umbral configurado, confirmando la eficacia de la firma implementada.



The image consists of two terminal window screenshots. The top window shows a series of alerts from a script named 'alert_dns_snooping.py'. Each alert indicates that a specific IP address (192.168.30.10, 192.168.30.20, and 192.168.30.1) has received 8 NXDOMAIN responses within a 10-second interval, which is above the configured threshold of 8. The alerts suggest a possible Kaminsky attack or subdomain mapping in progress. The bottom window shows a user executing a series of commands in a terminal. The user navigates to a directory, runs a Python script 'dns_snooping_attack.py' with specific arguments (resolver, domain, count, delay), and then uses 'find' and 'mv' commands to locate and move a screenshot file named 'captura3.png'.

```
adr3d0@adr3d0: ~/Laboratorio1-hacking-uem/P3_Mit...  
timos 10s (umbral: 8). Posible ataque de Kaminsky / mapeo de subdominios en cu  
rso.  
[ALERTA DNS SNOOPING] 192.168.30.10 ha recibido 8 respuestas NXDOMAIN en los u  
ltimos 10s (umbral: 8). Posible ataque de Kaminsky / mapeo de subdominios en c  
urso.  
[ALERTA DNS SNOOPING] 192.168.30.10 ha recibido 8 respuestas NXDOMAIN en los u  
ltimos 10s (umbral: 8). Posible ataque de Kaminsky / mapeo de subdominios en c  
urso.  
[ALERTA DNS SNOOPING] 192.168.30.20 ha recibido 8 respuestas NXDOMAIN en los u  
ltimos 10s (umbral: 8). Posible ataque de Kaminsky / mapeo de subdominios en c  
urso.  
[ALERTA DNS SNOOPING] 192.168.30.1 ha recibido 8 respuestas NXDOMAIN en los ul  
timos 10s (umbral: 8). Posible ataque de Kaminsky / mapeo de subdominios en cu  
rso.  
[ALERTA DNS SNOOPING] 192.168.30.10 ha recibido 8 respuestas NXDOMAIN en los u  
ltimos 10s (umbral: 8). Posible ataque de Kaminsky / mapeo de subdominios en c  
urso.  
[  
  
adr3d0@adr3d0: ~/Laboratorio1-hacking-uem/P3_Mit...  
ver  
  
(adr3d0@adr3d0)-[~/Laboratorio1-hacking-uem/P3_Mitm_y_suplantacion/doc]  
$ cd ~/Laboratorio1-hacking-uem/P3_Mitm_y_suplantacion/src  
sudo $(which uv) run python3 dns_snooping_attack.py --resolver 192.168.30.20 -  
-domain lab.local --count 20 --delay 0.1  
[sudo] password for adr3d0:  
[*] Enviando 20 consultas a subdominios aleatorios de 'lab.local' hacia 192.16  
8.30.20...  
[*] Consultas enviadas: 20/20  
[+] Rafaga completada: 20 consultas a subdominios inexistentes enviadas.  
  
(adr3d0@adr3d0)-[~/Laboratorio1-hacking-uem/P3_Mitm_y_suplantacion/src]  
$ find ~/Pictures/Screenshots -iname "*captura3*" 2>/dev/null  
/home/adr3d0/Pictures/Screenshots/captura3.png  
  
(adr3d0@adr3d0)-[~/Laboratorio1-hacking-uem/P3_Mitm_y_suplantacion/src]  
$ mv ~/Pictures/Screenshots/captura3.png ~/Laboratorio1-hacking-uem/P3_Mitm_
```

Figura 3: Alertas generadas por alert_dns_snooping.py durante la ejecucion de la rafaga de consultas a subdominios aleatorios con dns_snooping_attack.py.

3 Resultados

A continuacion se resumen de forma tabulada los principales resultados de la investigacion.

3.1 ARP Spoofing

Host	MAC real	MAC tras el ataque
Victima (192.168.10.20) – entrada del router	02:42:c0:a8:0a:02	02:42:86:4c:b7:0b (atacante)
Router (192.168.10.2) – entrada de la victima	02:42:c0:a8:0a:14	02:42:86:4c:b7:0b (atacante)

Tabla 1: Tablas ARP antes y despues del ataque manual con scapy [3].

Metrica	Valor
Intervalo de reenvio del veneno	2 segundos
Ventana de deteccion (<code>alert_arp spoof</code>)	30 segundos
Alertas generadas durante el ataque	Continuas, una por IP y ciclo de deteccion
Falsos positivos antes del ataque	0

Tabla 2: Parametros y resultados del sistema de deteccion de ARP spoofing.

3.2 DNS Snooping / Kaminsky

Parametro	Valor
Umbral de alerta (<code>--threshold</code>)	8 respuestas NXDOMAIN
Ventana temporal (<code>--window</code>)	10 segundos
Consultas enviadas en la rafaga de prueba	20
Longitud de subdominio aleatorio	8 caracteres alfanumericos
Dominio base	lab.local

Tabla 3: Configuracion empleada en la validacion del sistema de deteccion DNS snooping [3].

Codigo de respuesta DNS	Significado	Detectado como
NXDOMAIN (rcode=3)	Subdominio inexistente	Contabilizado hacia el umbral
NOERROR (rcode=0)	Subdominio existente	Ignorado por la firma (trafico legitimo)

Tabla 4: Codigos de respuesta DNS relevantes para la firma de deteccion implementada.

4 Conclusiones

Esta practica ha demostrado, en un laboratorio controlado, dos vectores de ataque que comparten una caracteristica de fondo: explotan protocolos de red (ARP y DNS) disenados en una epoca en la que la autentificacion fuerte de las respuestas no era una prioridad de diseno, confiando en su lugar en la buena fe de los participantes de la red.

El envenenamiento ARP demostro ser trivial de ejecutar a nivel tecnico – construir y reenviar periodicamente respuestas ARP falsificadas es una tarea de pocas lineas de codigo – y, sin embargo, extremadamente efectivo: en cuestion de segundos, tanto la victima como el router quedaron completamente engañados, redirigiendo su trafico mutuo a traves del atacante. La funcion de deteccion implementada, basada en la firma mas fiable disponible para este ataque (una misma IP respondiendo desde multiples MACs), demostro ser capaz de identificarlo de forma inmediata y sin falsos positivos en condiciones normales, confirmando que, aunque el

ataque es sencillo de ejecutar, también lo es de detectar si se monitoriza activamente el tráfico ARP de la red.

El caso de bettercap merece una reflexión aparte: la imposibilidad de hacerlo funcionar de forma fiable en este entorno concreto, pese a una investigación exhaustiva, ilustra un principio importante de la seguridad ofensiva: las herramientas de alto nivel, por muy potentes y populares que sean, dependen de asunciones sobre el entorno (detección de gateway, estabilidad de la cache interna de hosts) que pueden no cumplirse en todos los contextos, particularmente en redes virtualizadas o en contenedores. Comprender el mecanismo subyacente (en este caso, la construcción manual de paquetes ARP con scapy) resultó ser no solo una alternativa válida, sino una demostración de comprensión más profunda que depender únicamente de una herramienta de terceros.

En el caso del DNS snooping y el ataque de Kaminsky, la detección basada en umbral de respuestas NXDOMAIN demostró ser una firma sencilla pero efectiva para identificar rafagas de reconocimiento o intentos de envenenamiento de cache. La elección de un umbral y una ventana temporal adecuados resulta crítica: un umbral demasiado bajo generaría falsos positivos con tráfico DNS legítimo (errores tipográficos, aplicaciones mal configuradas), mientras que uno demasiado alto podría no detectar ataques más lentos y distribuidos en el tiempo, precisamente para evadir sistemas de detección como el implementado.

Desde una perspectiva de seguridad defensiva, ambos casos refuerzan la misma lección: la detección basada en anomalías de comportamiento (una IP con múltiples MACs, un volumen anormal de fallos de resolución) es más robusta frente a variaciones en la técnica de ataque que depender de firmas específicas de una herramienta concreta, ya que el patrón de comportamiento anómalo persiste independientemente de si el atacante usa bettercap, scapy, o cualquier otra implementación del mismo ataque subyacente.

5 Bibliografía

- [1] Docker Inc., «Docker Compose». [En línea]. Disponible en: <https://docs.docker.com/compose/>
- [2] Verzegnassi, Simone (evilsocket) and contributors, «bettercap – Swiss army knife for network attacks and monitoring». [En línea]. Disponible en: <https://www.bettercap.org/>
- [3] Biondi, Philippe and contributors, «Scapy – Python packet manipulation library». [En línea]. Disponible en: <https://scapy.net/>